WEB DEVELOPMENT LAB

Final Code – Experiments 1 to 9

Beginner-friendly code, file names, setup commands and testing instructions.

Contents

- 1. Personal Portfolio using HTML5, CSS Grid and Flexbox
- 2. JavaScript Quiz App with Timer and Score
- 3. Responsive Blog Template using Bootstrap 5
- 4. Weather Forecast Dashboard using Fetch API
- 5. Multi-page React Site using React Router
- 6. ToDo App using React Hooks and Local Storage
- 7. Angular 16 Contact Form using Reactive Forms
- 8. Book Management REST API using Express.js
- 9. Student Grade Summary using MongoDB Aggregation

Note: React and Angular experiments contain multiple files. Create each file at the path shown. Experiments 8 and 9 require their respective local services to be running.

Experiment 1: Personal Portfolio using HTML5, CSS Grid and Flexbox

Create a simple personal portfolio with Home, About, Skills and Contact sections. Save the following as index.html.

Setup / Run

- Create a folder named portfolio.
- Save the code below as index.html.
- Open index.html in a browser.

```
<!DOCTYPE html>
<html>
<head>
    <title>My Portfolio</title>
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 0;
            background: #f2f8fc;
            color: #222;
        }
        header {
            display: flex;
            justify-content: space-between;
            align-items: center;
            background: #d9efff;
            padding: 15px 25px;
            flex-wrap: wrap;
        }
        nav a {
            margin: 8px;
            color: #174a70;
            text-decoration: none;
            font-weight: bold;
        }
        section {
            padding: 25px;
            margin: 15px;
            background: white;
            border-radius: 8px;
        }
        .skills {
            display: grid;
            grid-template-columns: repeat(2, 1fr);
            gap: 12px;
        }
        .skills div {
            background: #b8e1f0;
            padding: 18px;
            border-radius: 5px;
        }
        footer {
            text-align: center;
            background: #174a70;
            color: white;
            padding: 15px;
        }
        @media (max-width: 600px) {
            header { flex-direction: column; }
            .skills { grid-template-columns: 1fr; }
        }
    </style>
</head>
<body>
    <header>
        <h1>Portfolio</h1>
        <nav>
```

```
            <a href="#home">Home</a>
            <a href="#about">About</a>
            <a href="#skills">Skills</a>
            <a href="#contact">Contact</a>
        </nav>
    </header>

    <section id="home">
        <h2>Hello!</h2>
        <p>Welcome to my portfolio.</p>
    </section>

    <section id="about">
        <h2>About Me</h2>
        <p>I am a Computer Science student interested in web development.</p>
    </section>

    <section id="skills">
        <h2>Skills</h2>
        <div class="skills">
            <div>Java</div>
            <div>HTML</div>
            <div>CSS</div>
            <div>JavaScript</div>
        </div>
    </section>

    <section id="contact">
        <h2>Contact Me</h2>
        <p>Email: your-email@gmail.com</p>
    </section>

    <footer>
        <p>Copyright &copy; 2026</p>
    </footer>
</body>
</html>
```

Experiment 2: JavaScript Quiz App with Timer and Scoring

Save this as index.html. The quiz displays questions, checks answers, counts marks and ends when all questions are answered or the timer reaches zero.

Setup / Run

- Create a folder named quiz-app.
- Save the code as index.html.
- Open it in a browser.

```
<!DOCTYPE html>
<html>
<head>
    <title>Quiz App</title>
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <style>
        body { font-family: Arial; background: #eaf6ff; padding: 25px; }
        .box { max-width: 500px; margin: auto; background: white; padding: 25px; border-radius: 10px; }
        button { padding: 10px 18px; margin: 6px; background: #3498db; color: white; border: 0; border-radius:
5px; cursor: pointer; }
        button:disabled { background: #aaa; }
    </style>
</head>
<body>
<div class="box">
    <h2>Quiz</h2>
    <p id="timer">Time: 30</p>
    <h3 id="question"></h3>
    <button id="a" onclick="check(0)"></button>
    <button id="b" onclick="check(1)"></button>
    <button id="c" onclick="check(2)"></button>
    <h3 id="result"></h3>
    <h3 id="score"></h3>
</div>

<script>
let questions = [
    {
        q: "Which language is used to create web page structure?",
        options: ["HTML", "C", "C++"],
        ans: 0
    },
    {
        q: "Which is a JavaScript library?",
        options: ["React", "C", "C++"],
        ans: 0
    }
];

let i = 0;
let marks = 0;
let time = 30;
let timer;

function load() {
    document.getElementById("question").innerHTML = questions[i].q;
    document.getElementById("a").innerHTML = questions[i].options[0];
    document.getElementById("b").innerHTML = questions[i].options[1];
    document.getElementById("c").innerHTML = questions[i].options[2];
}

function check(n) {
    if (i >= questions.length) return;

    if (n === questions[i].ans) {
        marks++;
    }

    i++;
```

```
    if (i < questions.length) {
        load();
    } else {
        endQuiz();
    }
}

function endQuiz() {
    clearInterval(timer);
    document.getElementById("question").innerHTML = "Quiz Completed!";
    document.getElementById("score").innerHTML =
        "Score: " + marks + " / " + questions.length;
    document.getElementById("a").style.display = "none";
    document.getElementById("b").style.display = "none";
    document.getElementById("c").style.display = "none";
}

timer = setInterval(function() {
    time--;
    document.getElementById("timer").innerHTML = "Time: " + time;

    if (time <= 0) {
        endQuiz();
    }
}, 1000);

load();
</script>
</body>
</html>
```

Experiment 3: Responsive Blog Template using Bootstrap 5

Save this as index.html. Bootstrap is loaded from a CDN, so an internet connection is required when opening the page.

Setup / Run

- Create a folder named blog.
- Save the code as index.html.
- Open it in a browser with internet access.

```
<!DOCTYPE html>
<html>
<head>
    <title>Blog</title>
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body>
    <nav class="navbar navbar-dark bg-dark navbar-expand-lg">
        <div class="container">
            <a class="navbar-brand" href="#">Blog</a>
            <button class="navbar-toggler" data-bs-toggle="collapse" data-bs-target="#menu">
                <span class="navbar-toggler-icon"></span>
            </button>
            <div class="collapse navbar-collapse" id="menu">
                <ul class="navbar-nav ms-auto">
                    <li class="nav-item"><a class="nav-link" href="#home">Home</a></li>
                    <li class="nav-item"><a class="nav-link" href="#technology">Technology</a></li>
                    <li class="nav-item"><a class="nav-link" href="#science">Science</a></li>
                    <li class="nav-item"><a class="nav-link" href="#contact">Contact</a></li>
                </ul>
            </div>
        </div>
    </nav>

    <div class="container mt-4" id="home">
        <h2 class="text-center">Latest Blogs</h2>

        <div class="card p-3 mt-3" id="technology">
            <h4>Technology</h4>
            <p>Latest updates in technology.</p>
            <button class="btn btn-primary">Read More</button>
        </div>

        <div class="card p-3 mt-3" id="science">
            <h4>Science</h4>
            <p>New discoveries in science.</p>
            <button class="btn btn-primary">Read More</button>
        </div>
    </div>

    <footer class="bg-dark text-white text-center p-3 mt-5" id="contact">
        Contact Us | My Blog
    </footer>

    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>
```

Experiment 4: Weather Forecast Dashboard using JavaScript Fetch API

Save this as index.html. This uses the free Open-Meteo Geocoding and Forecast APIs; no API key is required.

Setup / Run

- Create a folder named weather-app.
- Save the code as index.html.
- Open it in a browser with internet access.
- Enter a city name and click Get Weather.

```
<!DOCTYPE html>
<html>
<head>
    <title>Weather Dashboard</title>
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <style>
        body { font-family: Arial; background: #eaf6ff; margin: 0; padding: 25px; }
        .container { max-width: 800px; margin: auto; background: white; padding: 25px; border-radius: 12px;
text-align: center; }
        h2 { color: #2874a6; }
        input { padding: 10px; width: 55%; border: 1px solid gray; border-radius: 5px; }
        button { padding: 10px 18px; background: #3498db; color: white; border: 0; border-radius: 5px; cursor:
pointer; }
        #result { background: #dff2ff; padding: 12px; margin-top: 20px; border-radius: 8px; }
        .weather-box { display: flex; justify-content: center; gap: 15px; margin: 15px 0; }
        .box, .day { background: #eaf6ff; padding: 15px; border-radius: 8px; }
        #forecast { display: grid; grid-template-columns: repeat(3, 1fr); gap: 12px; margin-top: 15px; }
        @media(max-width: 600px) {
            #forecast { grid-template-columns: repeat(2, 1fr); }
            .weather-box { flex-direction: column; }
            input { width: 90%; margin-bottom: 10px; }
        }
    </style>
</head>
<body>
<div class="container">
    <h2>Weather Dashboard</h2>
    <p>Check the weather of your city</p>
    <input type="text" id="city" placeholder="Enter city name">
    <button onclick="getWeather()">Get Weather</button>

    <div id="result"></div>
    <h3>Next 5 Days Forecast</h3>
    <div id="forecast"></div>
</div>

<script>
async function getWeather() {
    let city = document.getElementById("city").value.trim();

    if (city === "") {
        document.getElementById("result").innerHTML = "Please enter a city name";
        return;
    }

    try {
        let url = "https://geocoding-api.open-meteo.com/v1/search?name="
                  + encodeURIComponent(city) + "&count=1";

        let response = await fetch(url);
        let data = await response.json();

        if (!data.results || data.results.length === 0) {
            document.getElementById("result").innerHTML = "City not found";
            document.getElementById("forecast").innerHTML = "";
            return;
        }
```

```
        let lat = data.results[0].latitude;
        let lon = data.results[0].longitude;
        let cityName = data.results[0].name;

        let weatherURL = "https://api.open-meteo.com/v1/forecast?latitude="
            + lat + "&longitude=" + lon
            + "&current=temperature_2m,relative_humidity_2m"
            + "&daily=temperature_2m_max,temperature_2m_min"
            + "&forecast_days=5&timezone=auto";

        let res = await fetch(weatherURL);
        let weather = await res.json();

        document.getElementById("result").innerHTML =
            "<h3>" + cityName + "</h3>" +
            "<div class='weather-box'>" +
            "<div class='box'><h3>Temperature</h3><h2>"
            + weather.current.temperature_2m + " °C</h2></div>" +
            "<div class='box'><h3>Humidity</h3><h2>"
            + weather.current.relative_humidity_2m + " %</h2></div>" +
            "</div>";

        let output = "";
        for (let i = 0; i < weather.daily.time.length; i++) {
            output += "<div class='day'><h4>" + weather.daily.time[i] + "</h4>"
                + "<p>Max: " + weather.daily.temperature_2m_max[i] + " °C</p>"
                + "<p>Min: " + weather.daily.temperature_2m_min[i] + " °C</p></div>";
        }

        document.getElementById("forecast").innerHTML = output;
    } catch (error) {
        document.getElementById("result").innerHTML = "Unable to get weather. Check your internet connection.";
        console.log(error);
    }
}
</script>
</body>
</html>
```

Experiment 5: Multi-page Website using React Router

Create a Vite React project and install React Router. The app uses separate components for Home, About and Contact.

Setup / Run

- npm create vite@latest app -- --template react
- cd app
- npm install
- npm install react-router-dom
- Create the files below inside src.
- Run using npm run dev.

File: src/App.jsx

```
import React from "react";
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";
import Home from "./Home";
import About from "./About";
import Contact from "./Contact";

function App() {
    return (
        <BrowserRouter>
            <div>
                <h3>Welcome</h3>
                <nav>
                    <Link to="/">Home</Link> |{" "}
                    <Link to="/about">About</Link> |{" "}
                    <Link to="/contact">Contact</Link>
                </nav>

                <Routes>
                    <Route path="/" element={<Home />} />
                    <Route path="/about" element={<About />} />
                    <Route path="/contact" element={<Contact />} />
                </Routes>
            </div>
        </BrowserRouter>
    );
}

export default App;
```

File: src/Home.jsx

```
function Home() {
    return (
        <div>
            <h2>Welcome to Home</h2>
            <p>This is Home</p>
        </div>
    );
}
export default Home;
```

File: src/About.jsx

```
function About() {
    return (
        <div>
            <h2>About Us</h2>
            <p>This is About.</p>
        </div>
    );
}
export default About;
```

File: src/Contact.jsx

```
function Contact() {
    return (
        <div>
            <h2>Contact</h2>
            <p>Email: your-email@gmail.com</p>
        </div>
    );
}
export default Contact;
```

File: src/main.jsx

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App.jsx";
import "./index.css";

ReactDOM.createRoot(document.getElementById("root")).render(
    <React.StrictMode>
        <App />
    </React.StrictMode>
);
```

File: src/index.css

```
body {
    font-family: Arial;
    margin: 30px;
    background-color: #f0f8ff;
}
nav {
    background-color: lightblue;
    padding: 15px;
}
nav a {
    text-decoration: none;
    color: black;
    margin: 15px;
    font-weight: bold;
}
nav a:hover {
    color: blue;
}
```

Experiment 6: ToDo App using React Hooks and Local Storage

Use useState to manage tasks and filters, and useEffect to save tasks in browser local storage. Create a Vite React project.

Setup / Run

- npm create vite@latest todoapp -- --template react
- cd todoapp
- npm install
- Replace src/App.jsx and src/index.css with the code below.
- Run using npm run dev.

File: src/App.jsx

```
import { useState, useEffect } from "react";

function App() {
    const [task, setTask] = useState("");
    const [tasks, setTasks] = useState(() => {
        return JSON.parse(localStorage.getItem("tasks")) || [];
    });
    const [filter, setFilter] = useState("All");

    useEffect(() => {
        localStorage.setItem("tasks", JSON.stringify(tasks));
    }, [tasks]);

    function addTask() {
        if (task.trim() !== "") {
            setTasks([...tasks, { text: task, done: false }]);
            setTask("");
        }
    }

    function toggleTask(index) {
        setTasks(tasks.map((t, i) =>
            i === index ? { ...t, done: !t.done } : t
        ));
    }

    function deleteTask(index) {
        setTasks(tasks.filter((t, i) => i !== index));
    }

    return (
        <div className="app">
            <h2>Todo App</h2>

            <input
                value={task}
                onChange={e => setTask(e.target.value)}
                placeholder="Enter task"
            />
            <button onClick={addTask}>Add</button>

            <div>
                <button onClick={() => setFilter("All")}>All</button>
                <button onClick={() => setFilter("Active")}>Active</button>
                <button onClick={() => setFilter("Done")}>Done</button>
            </div>

            {tasks.map((t, i) => (
                (filter === "All" ||
                 (filter === "Active" && !t.done) ||
                 (filter === "Done" && t.done)) &&

                <p key={i}>
                    <input
                        type="checkbox"
```

```
                        checked={t.done}
                        onChange={() => toggleTask(i)}
                    />
                    <span className={t.done ? "completed" : ""}>{t.text}</span>
                    <button onClick={() => deleteTask(i)}>Delete</button>
                </p>
            ))}
        </div>
    );
}

export default App;
```

File: src/index.css

```
body {
    font-family: Arial;
    background-color: #eaf6ff;
    margin: 30px;
}
.app {
    max-width: 600px;
    margin: auto;
    background: white;
    padding: 20px;
    border-radius: 10px;
}
h2 { color: #2874a6; }
input { padding: 8px; margin: 5px; }
button {
    background-color: lightblue;
    border: none;
    padding: 8px 15px;
    margin: 5px;
    cursor: pointer;
    border-radius: 5px;
}
.completed {
    text-decoration: line-through;
    color: gray;
}
```

The Vite-generated src/main.jsx can remain unchanged.

Experiment 7: Angular 16 Contact Form using Reactive Forms

Create an Angular 16 project using NgModule (non-standalone) and ReactiveFormsModule. This version validates required fields and email format.

Setup / Run

- npm install -g @angular/cli@16
- ng new contact-app --no-standalone
- Choose CSS when asked.
- cd contact-app
- Replace the files below.
- Run using ng serve --open.

File: src/app/app.module.ts

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { ReactiveFormsModule } from '@angular/forms';
import { AppComponent } from './app.component';

@NgModule({
    declarations: [AppComponent],
    imports: [
        BrowserModule,
        ReactiveFormsModule
    ],
    providers: [],
    bootstrap: [AppComponent]
})
export class AppModule { }
```

File: src/app/app.component.ts

```
import { Component } from '@angular/core';
import {
    FormGroup,
    FormControl,
    Validators
} from '@angular/forms';

@Component({
    selector: 'app-root',
    templateUrl: './app.component.html'
})
export class AppComponent {
    form = new FormGroup({
        name: new FormControl('', Validators.required),
        email: new FormControl('', [
            Validators.required,
            Validators.email
        ]),
        subject: new FormControl('', Validators.required),
        message: new FormControl('', Validators.required)
    });

    submitForm() {
        if (this.form.invalid) {
            alert("Please fill all fields correctly");
            return;
        }

        alert("Submitted successfully!");
        this.form.reset();
    }
}
```

File: src/app/app.component.html

```
<h2>Contact Form</h2>

<form [formGroup]="form" (ngSubmit)="submitForm()">
    <input formControlName="name" placeholder="Name">
    <br><br>

    <input formControlName="email" placeholder="Email">
    <br><br>

    <input formControlName="subject" placeholder="Subject">
    <br><br>

    <textarea formControlName="message" placeholder="Message"></textarea>
    <br><br>

    <button type="submit" [disabled]="form.invalid">
        Submit
    </button>
</form>
```

File: src/styles.css

```
body {
    font-family: Arial, sans-serif;
    background-color: #eaf6ff;
    margin: 0;
    padding: 30px;
}
form {
    background-color: white;
    max-width: 350px;
    margin: 40px auto;
    padding: 30px;
    border-radius: 10px;
    box-shadow: 0 2px 10px lightgray;
}
h2 {
    text-align: center;
    color: #2874a6;
}
input, textarea {
    width: 100%;
    padding: 10px;
    margin-bottom: 15px;
    border: 1px solid #aaa;
    border-radius: 5px;
    box-sizing: border-box;
    font-family: Arial;
}
button {
    width: 100%;
    padding: 12px;
    background-color: #3498db;
    color: white;
    border: none;
    border-radius: 5px;
}
button:disabled {
    background-color: #aaa;
}
@media (max-width: 500px) {
    form { margin: 20px auto; }
}
```

Experiment 8: Book Management REST API using Express.js

Create a REST API with GET, POST, PUT and DELETE routes. The book data is stored in an in-memory array for this introductory practical.

Setup / Run

- mkdir bookapi
- cd bookapi
- npm init -y
- npm install express
- Save the code as server.js.
- Run node server.js. Keep this terminal open.
- Open another CMD window and run the curl tests below.

File: server.js

```
const express = require('express');
const app = express();

app.use(express.json());

let books = [];

// Home route
app.get('/', (req, res) => {
    res.send('Welcome to Book API');
});

// Get all books
app.get('/books', (req, res) => {
    res.json(books);
});

// Add a book
app.post('/books', (req, res) => {
    if (Array.isArray(req.body)) {
        books.push(...req.body);
    } else {
        books.push(req.body);
    }
    res.send('Book added');
});

// Update a book using its array index
app.put('/books/:id', (req, res) => {
    const id = Number(req.params.id);

    if (!Number.isInteger(id) || id < 0 || id >= books.length) {
        return res.status(404).send('Book not found');
    }

    books[id] = { ...books[id], ...req.body };
    res.send('Book updated');
});

// Delete a book using its array index
app.delete('/books/:id', (req, res) => {
    const id = Number(req.params.id);

    if (!Number.isInteger(id) || id < 0 || id >= books.length) {
        return res.status(404).send('Book not found');
    }

    books.splice(id, 1);
    res.send('Book deleted');
});

app.listen(3000, () => {
```

```
    console.log('Running: http://localhost:3000');
});
```

CMD testing commands

```
:: GET - view all books
curl http://localhost:3000/books

:: POST - add a book
curl -X POST http://localhost:3000/books -H "Content-Type: application/json" -d
"{\"title\":\"Java\",\"author\":\"James\"}"

:: PUT - update book at index 0
curl -X PUT http://localhost:3000/books/0 -H "Content-Type: application/json" -d "{\"title\":\"Advanced
Java\",\"author\":\"James\"}"

:: DELETE - delete book at index 0
curl -X DELETE http://localhost:3000/books/0

:: GET - verify remaining books
curl http://localhost:3000/books
```

Note: This simple array-based API loses all book data when the server restarts. The id in the URL is the array index, not a database ID.

Experiment 9: MongoDB Aggregation to Summarize Student Grades

Use the MongoDB Node.js driver. Store each student's subject marks in an array, then use $unwind and $group to calculate total, average, highest and lowest marks.

Setup / Run

- Make sure MongoDB Server is installed and running locally on port 27017.
- mkdir studentdb
- cd studentdb
- npm init -y
- npm install mongodb
- Save the code as app.js.
- Run node app.js.

Schema design

```
{
    "stu_id": 1,
    "name": "Ravi",
    "dept": "CSE",
    "marks": [80, 85, 70]
}
```

The students collection contains stu_id (number), name (string), dept (string) and marks (array of numbers).

File: app.js

```
const { MongoClient } = require('mongodb');

async function main() {
    const client = new MongoClient('mongodb://127.0.0.1:27017');

    try {
        await client.connect();
        console.log("MongoDB Connected");

        const db = client.db('college');
        const stu = db.collection('students');

        // Clear old practical records before inserting sample data
        await stu.deleteMany({});

        await stu.insertMany([
            {
                stu_id: 1,
                name: "Ravi",
                dept: "CSE",
                marks: [80, 85, 70]
            },
            {
                stu_id: 2,
                name: "Swathi",
                dept: "CSE",
                marks: [90, 95, 85]
            },
            {
                stu_id: 3,
                name: "Kiran",
                dept: "ECE",
                marks: [75, 80, 65]
            }
        ]);

        console.log("Students inserted");

        const result = await stu.aggregate([
```

```
            { $unwind: "$marks" },
            {
                $group: {
                    _id: "$name",
                    totalMarks: { $sum: "$marks" },
                    avgMarks: { $avg: "$marks" },
                    maxMarks: { $max: "$marks" },
                    minMarks: { $min: "$marks" }
                }
            },
            { $sort: { _id: 1 } }
        ]).toArray();

        console.log("Student Grade Summary:");
        console.table(result);

    } catch (error) {
        console.log(error);
    } finally {
        await client.close();
    }
}

main();
```

Expected summary

Student	Total	Average	Maximum	Minimum
Ravi	235	78.33	85	70
Swathi	270	90.00	95	85
Kiran	220	73.33	80	65

Quick Run Reference

Experiment	How to run
Experiment 1	Open index.html in a browser.
Experiment 2	Open index.html in a browser.
Experiment 3	Open index.html in a browser with internet.
Experiment 4	Open index.html in a browser with internet.
Experiment 5	npm run dev (inside Vite React project).
Experiment 6	npm run dev (inside Vite React project).
Experiment 7	ng serve --open (inside Angular project).
Experiment 8	node server.js (inside bookapi).
Experiment 9	node app.js (with MongoDB running).

Before the practical: check the file names carefully, start required local servers (Angular/Express/MongoDB), and keep the terminal window running where required.